

AutoEmpirical MAS 架构设计报告 v2

基于数据集补全后的最新统计（stage1_enriched.csv）、现有方法局限性分析与文献综述，对 Stage 2（Bug Commit 提取）和 Stage 3（多标签分类）的多智能体系统进行专项设计。本文档同时服务于论文写作与代码实现两个层面。

一、数据集现状分析（基于 stage1_enriched.csv）

1.1 整体规模

经 GitHub API 补全后，数据集从 53.9% 的 body 可用率提升至 **94.8%**，显著改善了可用于自动标注的文本质量。

指标	原始 stage1_unified_labels.csv	补全后 stage1_enriched.csv
总记录数	1,907	1,907
有 body	1,027 (53.9%)	1,808 (94.8%)
有 comments	680 (35.7%)	1,454 (76.2%)
body + comments 均有	675 (35.4%)	1,437 (75.4%)
仅有 title	875 (45.9%)	99 (5.2%)

剩余 99 条无法补全的记录中，60 条来自 fse2023 的 StackOverflow 链接（非 GitHub 来源），39 条为 GitHub 私有仓库或已删除 issue，属于数据来源层面的不可逆缺失。

1.2 标签完整性

标签字段	非空记录数	占比	说明
root_cause	1,907	100%	所有论文均提供
symptom	1,869	98.0%	极少量缺失
bug_type	1,573	82.5%	部分论文未标注
fix_type	1,154	60.5%	缺失率最高，是分类难点
全 4 项均有	784	41.1%	可用于完整监督评估的子集

按论文的标签覆盖率差异极大：

paper_id	全 4 项覆盖率	body 可用率	特征
icse2022_wechat	100%	100%	最完整
ase2022	92%	99%	高质量
ase2021	75%	100%	标签部分缺失
fse2021	0%	99%	无 bug_type/fix_type
fse2023	0%	76%	无 bug_type/fix_type
icse2022_perf	0%	100%	无 bug_type/fix_type
icse2023	0%	100%	无 bug_type/fix_type
icse2024	0%	100%	无 bug_type/fix_type

1.3 标签分布与 Taxonomy 异构性

高频 symptom 标签（来自不同论文，命名不统一）：

标签值	出现次数	来源论文
Crash	616	ase2022
Incorrect Functionality	186	ase2022
Build & Initialization Failure	141	ase2022
performance bug	137	ase2022
Poor Performance	114	ase2022
Functional error	62	icse2024

高频 root_cause 标签（同样存在命名异构）：

标签值	出现次数	来源论文
Incorrect Code Logic	159	ase2022
API Misuse	73	ase2022
Dependency Error	61	ase2022
Logic error	50	icse2022_wechat
Inconsistency	49	icse2022_wechat

关键发现： `Logic error` (icse2022_wechat) 与 `Incorrect Code Logic` (ase2022) 语义相同但拼写不同； `performance bug` 与 `Poor Performance` 指向同一概念。这种跨论文的命名异构性是 Stage 3 需要重点解决的问题。

1.4 文本质量分析

各论文 body 的平均长度揭示了文本质量的差异：

paper_id	平均 body 长度	质量评估
icse2022_perf	2,726 字符	丰富，有详细描述
icse2023	1,999 字符	丰富
ase2021	1,622 字符	中等
ase2022	1,491 字符	中等
fse2023	1,128 字符	中等（含 SO 链接）
icse2022_wechat	64 字符	极短（通常仅为 URL）
icse2024	67 字符	极短
fse2021	41 字符	极短

新发现： 补全之后，icse2022_wechat、icse2024、fse2021 虽然 body 可用率高，但平均长度极短（41-67 字符），说明这三个论文的 "body" 实际上只是 issue URL 或极简描述，并非真正的 issue 正文。这意味着 Stage 2/3 的处理难度不只是"有没有 body"，还包括 "body 是否有实质内容"。

二、现有方法的局限性

基于对 17 篇相关文献的系统梳理，现有自动化标注方法存在以下核心缺陷，本设计针对每一条提出对应的解决方案。

2.1 通用局限

ID	局限描述	文献证据	严重程度
L1	自动标注假阳性率高达 8-54%，单模型无校验机制	Dunivin et al., arXiv:2601.09905	● 高
L2	LLM 输出非确定性：同 prompt 不同运行产生不同标签，无一致性追踪	OLAF, arXiv:2512.15979	● 高

ID	局限描述	文献证据	严重程度
L3	标注数据集本身含约 33% 错误标签（feature request 被标为 bug 等）	Laiq & Dobslaw, arXiv:2505.01469	● 高
L4	模糊/不完整 issue 被当作噪声丢弃，而非显式标记处理	Gon et al., arXiv:2605.17561	● 中
L5	线性流水线的级联错误：上游错误无法被下游纠正	Dunivin et al., arXiv:2601.09905	● 中
L6	LLM 在需要因果推理的 root cause 分析上显著弱于领域专家	Martinez, arXiv:2510.18456	● 中

2.2 Stage 2（Bug 有效性验证）的特有局限

ID	局限描述	文献证据
L2a	过度依赖 issue 文本，忽略 commit/PR 等外部链接证据	Laiq, arXiv:2505.01469
L2b	无法区分"被报告的问题"与"真实 bug"，倾向将所有 issue 判为 bug	Andrade et al., arXiv:2503.00660
L2c	invalid/wont-fix 的细粒度子类判断能力极差（F1 低至 0.00–0.29）	Gon et al., arXiv:2605.17561
L2d	缺乏不确定性量化，永远输出确定性判断	OLAF, arXiv:2512.15979

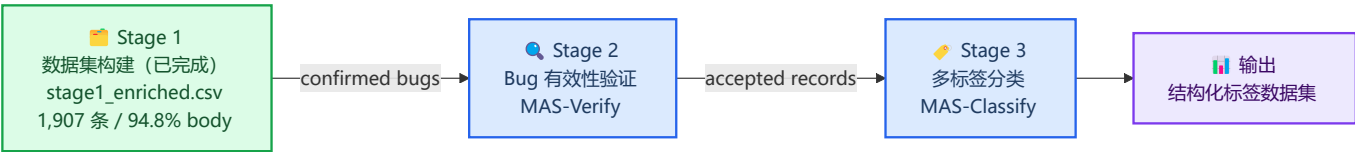
2.3 Stage 3（多标签分类）的特有局限

ID	局限描述	文献证据
L3a	单视角分类导致 symptom 与 root_cause 的因果逻辑不一致	本项目现有 pipeline.py
L3b	Taxonomy 定义仅作文本提示，LLM 可生成 taxonomy 外的幻觉标签	Flow-of-Action, arXiv:2502.08224
L3c	无 Inter-Rater Reliability（IRR）机制，无法模拟人工实证研究的质控流程	Díaz et al., arXiv:2107.11449
L3d	Critic 做事后检查而非预防性质疑，无法影响分类过程本身	现有 pipeline.py
L3e	多 LLM 在温度和 persona 参数下对 consensus 影响极大，但现有系统未建模此变异	Borchers et al., arXiv:2507.11198

ID	局限描述	文献证据
L3f	LLM 的 disagreement 模式本身是有价值的过程数据，被现有系统完全忽略	Tajik et al., arXiv:2601.12618

三、总体架构与设计原则

3.1 三阶段流水线



3.2 核心设计原则

本设计遵循以下四条原则，每条原则均有对应的文献依据：

P1 — 证据驱动 (Evidence-Driven)

所有判断必须锚定于 issue 文本中可引用的具体证据，而非 LLM 的参数化知识。灵感来源：From Threads to Trajectories (arXiv:2604.25880) 的多角色证据提取框架。

P2 — 多视角独立 (Multi-Perspective Independence)

不同角色从不同认知立场独立分析，防止单视角偏差和级联错误 (L5)。灵感来源：Multi-role Consensus (arXiv:2403.14274) 的双视角迭代讨论框架，F1 提升 16.13%。

P3 — IRR 驱动辩论 (IRR-Driven Debate)

模拟人工实证研究的 IRR 流程——独立标注 → 一致性检测 → 分歧驱动讨论 → 共识综合。灵感来源：Díaz et al. (arXiv:2107.11449) 的 SE 领域 IRR 方法论；AI Annotation Orchestration (arXiv:2511.09785) 证明 orchestration 可将 kappa 提升 58%。

P4 — SOP 约束 (SOP-Constrained)

Taxonomy 定义以标准操作程序 (SOP) 格式硬约束 LLM 的输出空间，防止幻觉标签 (L3b)。灵感来源：Flow-of-Action (arXiv:2502.08224) 在 RCA 任务中将准确率从 35.5% 提升至 64%。

四、Stage 2 详细设计：Bug 有效性验证 MAS

4.1 任务定义

输入: `stage1_enriched.csv` 中的一条 issue 记录 (title + body + comments + metadata)

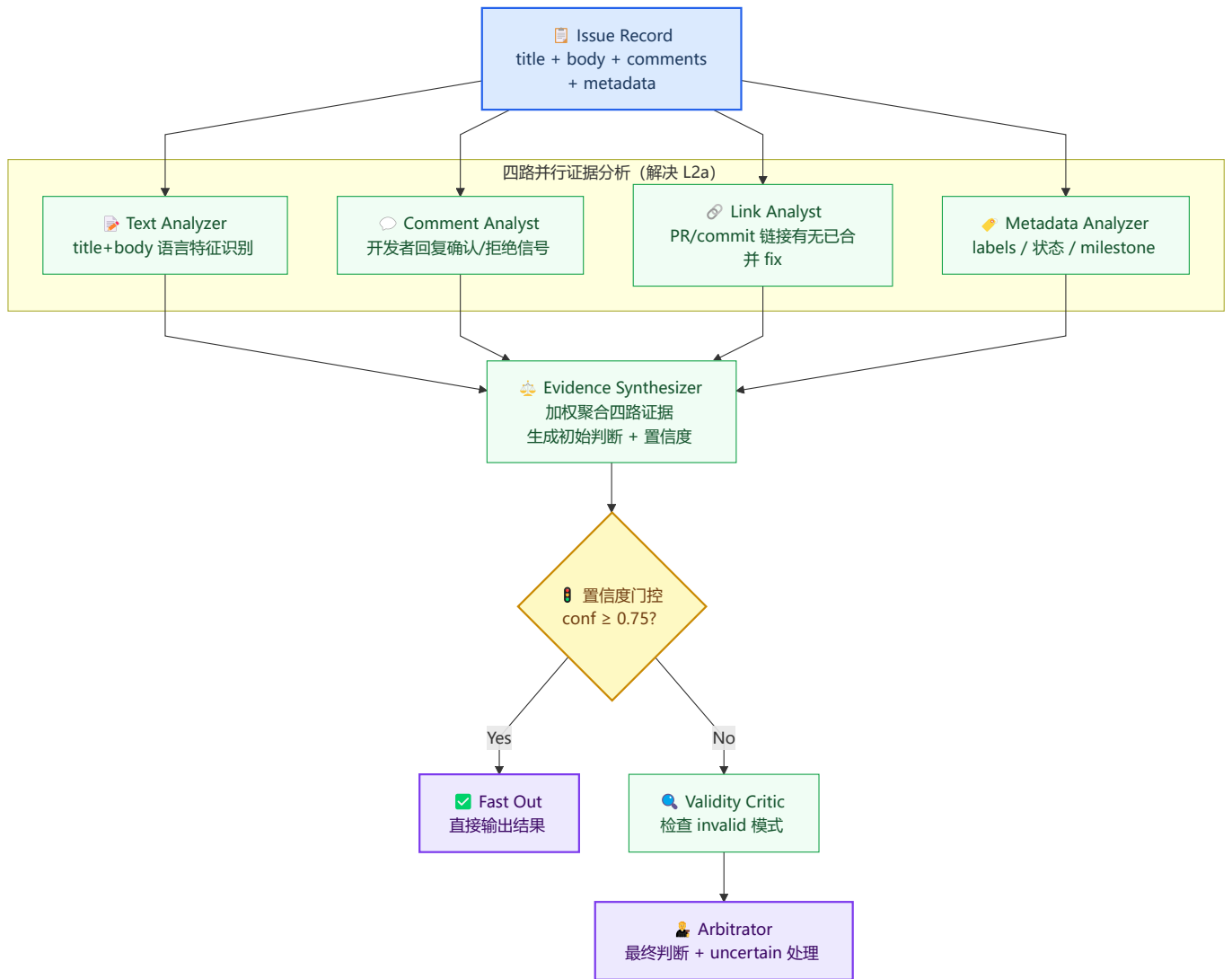
输出:

```
1  {
2    "record_id": "ase2022_001",
3    "verdict": "Accepted | Rejected | Uncertain",
4    "confidence": 0.88,
5    "evidence": ["Developer confirmed on 2024-01-10", "PR #456 merged with
6    fix"],
7    "linked_commits": ["https://github.com/.../commit/abc123"],
8    "invalid_pattern": null,
9    "review_flag": false,
10   "text_available": "full | short_body | title_only",
11   "api_calls": 4
12 }
```

解决的问题: L1 (高误报), L2 (非确定性), L4 (模糊 issue), L2a–L2d。

4.2 架构设计

Stage 2 的核心设计思想是**四路并行证据聚合 + 置信度门控**。四路证据相互独立, 任何一路都无法单独决定结果, Synthesizer 对证据进行加权聚合。



4.3 各 Agent 职责与 Prompt Schema

4.3.1 Text Analyzer

职责： 识别 title 和 body 中的 bug 报告语言特征，使用 SOP 步骤约束分析过程（P4 原则）。

System Prompt（SOP 格式，参考 Flow-of-Action arXiv:2502.08224）：

```
1 You are the Text Analyzer Agent. Follow these steps in order:
2
3 STEP 1 – Scan for bug-report signals:
4     Positive (bug): error messages, stack traces,
5     "crash/fail/error/wrong/unexpected",
6     "expected X but got Y", reproducible steps, version/environment info.
7     Negative (not-bug): "please add", "feature request", "how do I",
8     documentation issues.
9
10 STEP 2 – Account for short body:
11     If body length < 100 chars, downgrade confidence by 0.2 regardless of
12     signals.
13
14 STEP 3 – Output structured verdict.
15
16 Return valid JSON only.
```

Output Schema:

```
1 {
2   "bug_signals": ["'crash with OOM error'", "'stack trace attached'"],
3   "non_bug_signals": [],
4   "text_verdict": "likely_bug | likely_not_bug | ambiguous",
5   "confidence": 0.85,
6   "body_quality": "rich | short | url_only | absent"
7 }
```

对短 body 的处理（解决 1.4 节发现的新问题）： icse2022_wechat 等论文的 body 平均仅 64 字符，Text Analyzer 检测到 `body_quality=url_only` 时主动降低置信度，触发 Validity Critic 路径。

4.3.2 Comment Analyzer

职责： 从开发者评论中提取“已确认 bug”或“非 bug”的决定性线索，解决 L2a（过度依赖文本，忽略开发者确认信号）。

Output Schema:


```

1  {
2    "confirmation_signals": ["'confirmed by @maintainer'", "'fix merged in PR #456'"],
3    "rejection_signals": ["'working as intended'", "'this is not a bug'"],
4    "developer_verdict": "confirmed_bug | rejected | ambiguous | no_comments",
5    "confidence": 0.92,
6    "key_quote": "fix merged in #456 – closes this issue"
7  }

```

4.3.3 Link Analyzer

职责： 解析 issue 文本中的 PR/commit 链接，推断是否存在已合并的 bug fix，解决 L2a。有已合并 fix 是"确实是 bug"的强证据。

Output Schema:

```

1  {
2    "linked_prs": ["https://github.com/.../pull/456"],
3    "linked_commits": [],
4    "fix_evidence": "merged_fix | open_pr | no_fix | cannot_determine",
5    "confidence": 0.95
6  }

```

4.3.4 Metadata Analyzer

职责： 分析 GitHub issue 的结构化元数据，解决 L2b（将 feature request 误判为 bug）。

Output Schema:

```

1  {
2    "github_labels": ["bug", "confirmed"],
3    "issue_state": "closed",
4    "has_bug_label": true,
5    "has_wontfix_label": false,
6    "metadata_verdict": "likely_bug | likely_not_bug | ambiguous",
7    "confidence": 0.80
8  }

```

4.3.5 Evidence Synthesizer

职责： 加权聚合四路证据，生成初始判断和置信度估计。

权重设计（基于证据强度，解决 C1 text-only 问题）：

```
1  BASE_WEIGHTS = {
2      "link_evidence":      0.40, # 最强: 直接 fix 证明存在 bug
3      "developer_comment": 0.30, # 次强: 开发者明确确认
4      "metadata":          0.20, # 中等: GitHub label
5      "text_analysis":      0.10, # 最弱: 仅凭文本语言
6  }
7
8  # 当 body 极短 (url_only/absent) 时, text weight 进一步降低
9  if body_quality in ("url_only", "absent"):
10     weights["text_analysis"] = 0.05
11     # 剩余权重重分配给其他三路
```

Output Schema :

```
1  {
2      "synthesized_verdict": "Accepted",
3      "confidence": 0.87,
4      "evidence_summary": {
5          "text": {"verdict": "likely_bug", "weight": 0.10},
6          "comments": {"verdict": "confirmed_bug", "weight": 0.30},
7          "links": {"verdict": "merged_fix", "weight": 0.40},
8          "metadata": {"verdict": "likely_bug", "weight": 0.20}
9      },
10     "supporting_evidence": ["Developer confirmed", "PR #456 merged"],
11     "conflicting_evidence": []
12 }
```

4.3.6 Validity Critic

职责： 专项检查 invalid bug 的四种典型失败模式，解决 L2c (invalid 子类 F1 低至 0.00–0.29, Gon et al. arXiv:2605.17561)。

SOP 格式 Prompt（四种 invalid 模式枚举）：

```
1  You are the Validity Critic. Check ONLY for these four invalid-bug patterns:
2
```

```

3 | PATTERN A – Wrong Version/Environment:
4 |   Signal: issue resolved after updating dependency/runtime version
5 |   → Reject as: invalid_wrong_version
6 |
7 | PATTERN B – Usage Error:
8 |   Signal: developer says "this is by design", links to documentation
9 |   → Reject as: invalid_usage_error
10 |
11 | PATTERN C – Duplicate:
12 |   Signal: "duplicate of #xxx", or identical issue reference
13 |   → Reject as: invalid_duplicate
14 |
15 | PATTERN D – Feature Request Misabeled:
16 |   Signal: request language present, no observable error behavior
17 |   → Reject as: invalid_feature_request
18 |
19 | If none match: maintain current verdict unchanged.
20 | Return JSON only.

```

Output Schema:

```

1 | {
2 |   "invalid_pattern": "none | wrong_version | usage_error | duplicate |
   feature_request",
3 |   "revised_verdict": "Accepted | Rejected | Uncertain",
4 |   "revised_confidence": 0.90,
5 |   "evidence_for_revision": ["developer explicitly said 'working as designed'"]
6 | }

```

4.3.7 Arbitrator

职责： 综合 Synthesizer 和 Critic 的结果，输出最终判断。当置信度 < 0.45 或证据严重冲突时输出 **Uncertain**，解决 L2d（缺乏不确定性量化）。

Uncertain 触发条件：

```

1  def should_mark_uncertain(confidence: float, evidence_summary: dict) → bool:
2      if confidence < 0.45:
3          return True
4      # 有强正反证据同时存在 (如 link=merged_fix 但 comments=rejected)
5      strong_positive = evidence_summary["links"]["verdict"] == "merged_fix"
6      strong_negative = evidence_summary["comments"]["verdict"] == "rejected"
7      if strong_positive and strong_negative:
8          return True
9      return False

```

4.4 置信度门控参数

阈值 ≥ 0.75 的依据:

Dunivin et al. (arXiv:2601.09905) 报告自动标注假阳性率 8–54%，说明直接信任高置信度输出本身有风险。0.75 作为初始阈值，在 50 条开发集上通过 precision-recall 曲线确定最终值。高于此阈值的案例走 Fast Path（4 次 API 调用），低于此阈值的案例进入 Validity Critic 路径（6 次 API 调用）。

五、Stage 3 详细设计：多标签分类 MAS

5.1 任务定义

输入：Stage 2 输出的 `Accepted` 记录

输出：

```

1  {
2      "record_id": "ase2022_001",
3      "labels": {
4          "symptom": "Crash",
5          "root_cause": "Resource Management",
6          "bug_type": "Memory Bug",
7          "fix_type": "Code Change"
8      },
9      "uncertain_fields": [],
10     "consistency_score": 0.92,
11     "annotator_agreements": {
12         "symptom": 1.0, "root_cause": 0.67, "bug_type": 1.0, "fix_type": 1.0
13     },
14     "debate_rounds": 1,

```

```

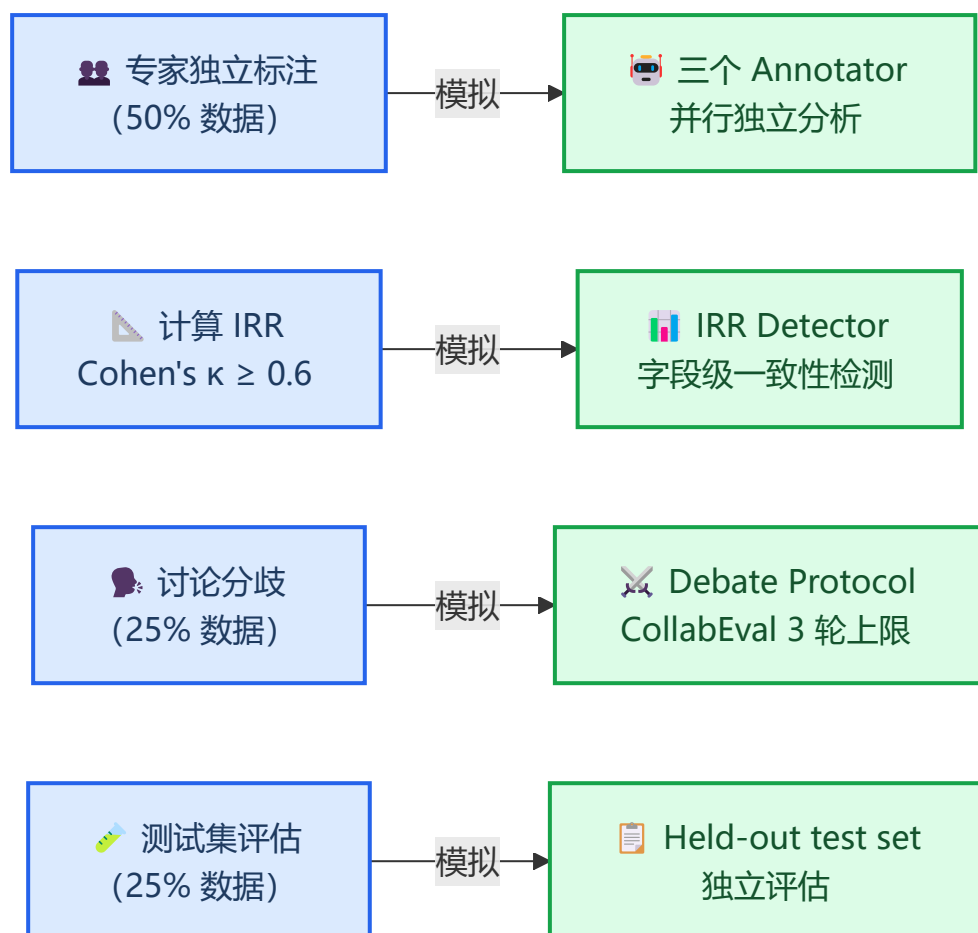
15     "debate_termination": "convergence",
16     "confidence": {"symptom": 0.95, "root_cause": 0.82, "bug_type": 0.88,
    "fix_type": 0.91}
17 }

```

解决的问题：L3a–L3f，以及数据集中的 taxonomy 异构性（见 1.3 节）。

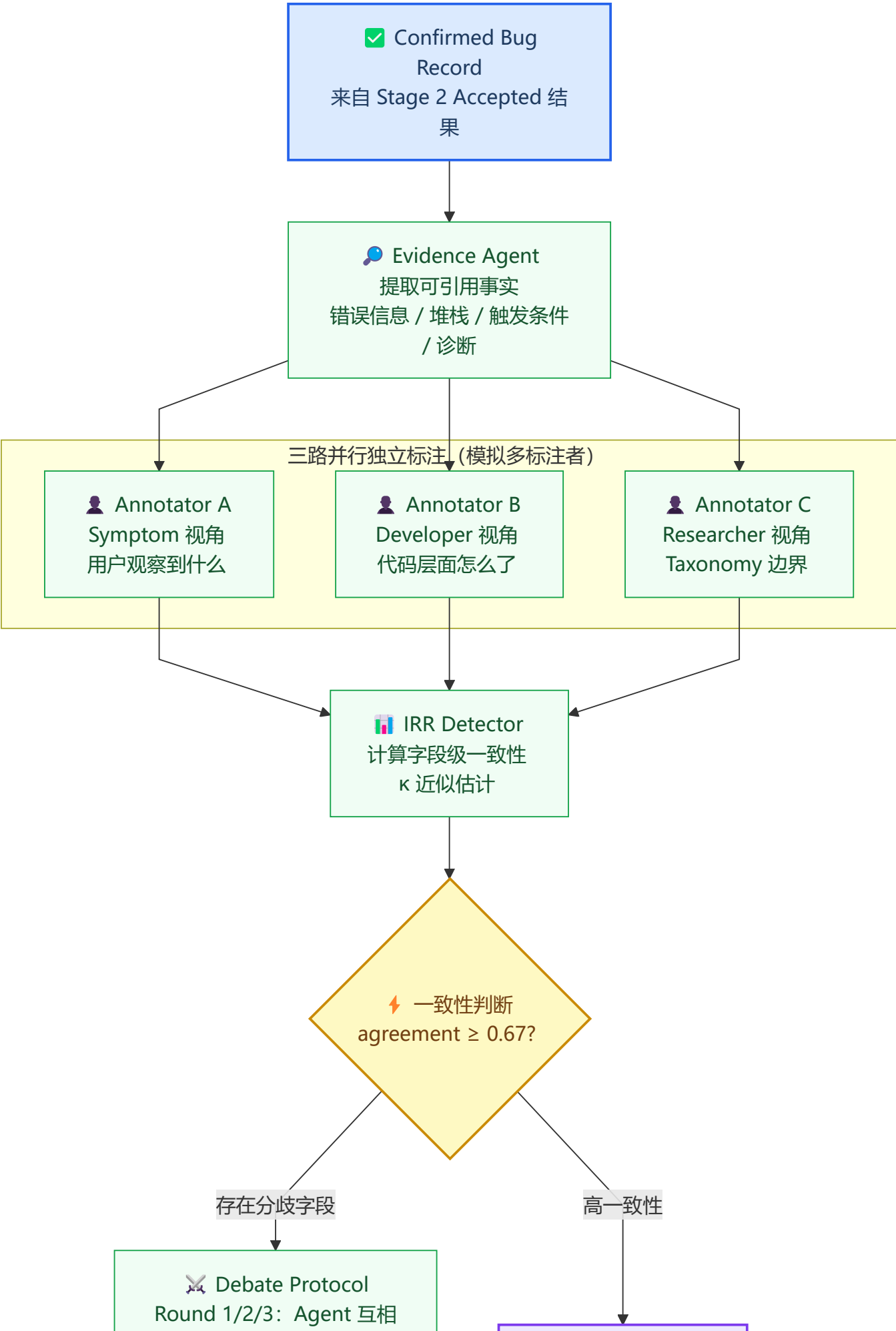
5.2 人工 IRR 流程的 MAS 模拟

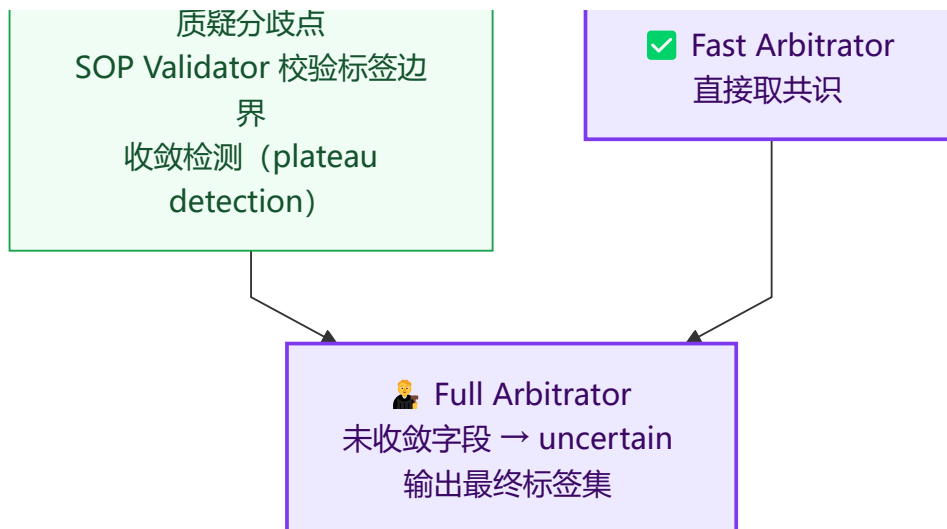
图如下：



AI Annotation Orchestration (arXiv:2511.09785) 的实证结果表明：引入 orchestration（让 verifier 检查 annotator 的标签）可将 Cohen's kappa 提升 58%，为本设计的 IRR-debate 机制提供了直接的实验支持。

5.3 架构设计





5.4 三个 Annotator 的 Prompt 设计

Annotator A: Symptom 视角

认知立场: "站在用户角度——系统表现出了什么可观测的异常行为？"

```
1 You are Annotator A: Symptom Perspective.
2 Focus ONLY on what the user observed. Do not reason about why it happened.
3
4 Symptom taxonomy (SOP):
5 A. Crash – program terminates unexpectedly / fatal exception / OOM
6 B. Incorrect Functionality – wrong output, wrong behavior, off-by-one
7 C. Build & Initialization Failure – fails to build, install, or import
8 D. Performance Degradation – slowdown, excessive memory, timeout
9 E. Hang – stops responding, infinite loop
10 F. Data Corruption – data modified incorrectly
11 G. Security Vulnerability – unintended security-relevant behavior
12
13 STEP 1: Identify observable symptoms from evidence package.
14 STEP 2: Map to exactly ONE primary category.
15 STEP 3: Cite the specific evidence phrase that supports your choice.
16
17 Return JSON only. Do not reveal reasoning beyond the JSON.
```

Output Schema:


```
1 {
2   "annotator": "A",
3   "symptom": "Crash",
4   "symptom_evidence": "ResourceExhaustedError: OOM when allocating tensor",
5   "confidence": 0.93,
6   "uncertain": false
7 }
```

Annotator B: Developer 视角

认知立场： "站在实现者角度——代码层面发生了什么导致了这个问题？ "

```
1 You are Annotator B: Developer Perspective.
2 Focus on the TECHNICAL ROOT CAUSE and how to fix it. Do not describe symptoms.
3
4 Root cause taxonomy (SOP):
5 A. Incorrect Code Logic – wrong algorithm, condition, or calculation
6 B. API Misuse – incorrect use of library/framework API
7 C. Dependency Error – wrong version, breaking change in upstream
8 D. Resource Management – memory leak, buffer overflow, file handle leak
9 E. Concurrency Issue – race condition, deadlock, data race
10 F. Configuration Error – wrong parameter, wrong environment setup
11 G. Type/Value Error – null pointer, index out of bounds, type mismatch
12
13 Fix type taxonomy:
14 A. Code Change – logic fix in source
15 B. Configuration Fix – parameter or config change
16 C. Version Update – upgrading dependency
17 D. Documentation Fix – clarifying usage
18 E. Workaround – temporary mitigation
19
20 STEP 1: Identify root cause from developer diagnosis in evidence.
21 STEP 2: Infer fix type from available hints.
22 STEP 3: Cite evidence.
23
24 Return JSON only.
```

Output Schema:

```

1  {
2      "annotator": "B",
3      "root_cause": "Resource Management",
4      "root_cause_evidence": "Memory pre-allocation issue in cuDNN backend",
5      "fix_type": "Code Change",
6      "fix_evidence": "PR #456 modifies memory allocation logic",
7      "confidence": 0.82,
8      "uncertain": false
9  }

```

Annotator C: Researcher 视角

认知立场： "站在实证研究者角度——这条记录在分类体系中属于哪一类？Annotator A/B 的标签是否彼此因果自治？ "

Annotator C 同时承担一致性仲裁的职责，直接应对 L3a（symptom-root_cause 不一致）和 L3b（taxonomy 外幻觉标签）：

```

1  You are Annotator C: Researcher Perspective.
2  Your dual role:
3      1. Determine bug_type from the research taxonomy
4      2. Check whether Annotator A and B's labels are causally consistent
5
6  Bug type taxonomy (SOP):
7  A. Semantic Bug – wrong semantics / incorrect behavior
8  B. Performance Bug – functionally correct but inefficient
9  C. Memory Bug – memory allocation or access error
10 D. Concurrency Bug – race condition or deadlock
11 E. Configuration Bug – wrong configuration causes failure
12 F. Security Bug – enables unintended access
13
14 Consistency check rules:
15 - Crash + Resource Management → CONSISTENT (common pattern)
16 - Incorrect Functionality + Incorrect Code Logic → CONSISTENT
17 - Crash + API Misuse → CONSISTENT (misuse can cause crashes)
18 - Performance Degradation + Incorrect Code Logic → NEEDS VERIFICATION
19   (logic errors don't typically cause perf issues; check evidence)
20 - Incorrect Functionality + Resource Management → INCONSISTENT
21   (resource issues cause crashes, not wrong output – flag for debate)
22
23 Return JSON only.

```

Output Schema:

```
1  {
2    "annotator": "C",
3    "bug_type": "Memory Bug",
4    "bug_type_evidence": "OOM error from memory pre-allocation classified as
Memory Bug in SE literature",
5    "consistency_check": {
6      "symptom_root_cause_consistent": true,
7      "explanation": "Crash (A) caused by Resource Management (B) is a well-
documented pattern",
8      "flag_for_debate": false
9    },
10   "confidence": 0.87,
11   "uncertain": false
12 }
```

5.5 IRR 检测协议

理论依据： Díaz et al. (arXiv:2107.11449) 在 SE 领域 IRR 研究中指出 Cohen's $\kappa \geq 0.6$ 为可接受阈值。本设计使用三个 Annotator 的字段级多数一致性作为 κ 的近似估计，计算成本低且在三分类场景下等价。

计算公式：

```
1  from collections import Counter
2
3  def field_agreement(annotations: list[dict], field: str) → float:
4      """三个 Annotator 对某字段的一致性: 多数标签出现次数 / 总数"""
5      values = [a[field] for a in annotations if not a.get("uncertain")]
6      if len(values) < 2:
7          return 0.0
8      most_common_count = Counter(values).most_common(1)[0][1]
9      return most_common_count / len(values)
10
11  DEBATE_THRESHOLD = 0.67 # 等价于"至少有1个与另2个不同"
12
13  fields_to_debate = [
14      f for f in ["symptom", "root_cause", "bug_type", "fix_type"]
15      if field_agreement(annotations, f) < DEBATE_THRESHOLD
16  ]
```

IRR Detector 输出示例:

```
1  {
2    "field_agreements": {
3      "symptom": 1.0,
4      "root_cause": 0.67,
5      "bug_type": 0.33,
6      "fix_type": 1.0
7    },
8    "fields_to_debate": ["bug_type"],
9    "fast_path_fields": ["symptom", "root_cause", "fix_type"],
10   "trigger_debate": true
11 }
```

5.6 辩论协议 (Debate Protocol)

设计依据: CollabEval (arXiv:2603.00993) 的三阶段协作评估框架, 以及 Disagreement as Data (arXiv:2601.12618) 的发现——LLM 的 disagreement 模式本身是有分析价值的过程数据 (L3f), 不应仅被视为需要消除的噪声。

辩论轮次结构

每轮辩论对每个 `fields_to_debate` 中的字段执行以下步骤:

1. **Annotator A** 陈述自己的立场 + 提出对 B/C 不同意见的质疑
2. **Annotator B** 回应, 可维持或修改立场
3. **Annotator C** 作为 Researcher, 引用 SOP 定义裁定双方立场的 taxonomy 合规性
4. **SOP Validator** 静默检查: 任何 Agent 提出的标签是否在 SOP 定义范围内

SOP Validator 的校验逻辑

```
1  def validate_label(label: str, field: str, sop: dict) → dict:
2      valid_labels = {cat["name"] for cat in sop[field]["categories"]}
3      aliases = sop.get("aliases", {}).get(field, {})
4
5      # 尝试别名归一化 (解决 1.3 节 taxonomy 异构性问题)
6      normalized = aliases.get(label.lower(), label)
7
8      if normalized not in valid_labels:
9          return {
10             "valid": False,
11             "reason": f"'{label}' not in taxonomy",
```

```

12         "suggested": find_closest(label, valid_labels)
13     }
14     return {"valid": True, "normalized_label": normalized}

```

Taxonomy 别名映射（解决数据集异构性，见 1.3 节）：

```

1  {
2      "aliases": {
3          "symptom": {
4              "performance bug": "Performance Degradation",
5              "poor performance": "Performance Degradation",
6              "functional error": "Incorrect Functionality",
7              "hang": "Hang",
8              "incorrect": "Incorrect Functionality",
9              "crash": "Crash"
10         },
11         "root_cause": {
12             "logic error": "Incorrect Code Logic",
13             "inconsistency": "Incorrect Code Logic",
14             "incompatibility between 3rd-party dl library and tf.js": "Dependency
Error"
15         }
16     }
17 }

```

收敛检测与终止条件

三条终止规则（来自 CollabEval arXiv:2603.00993）：

```

1  def check_termination(
2      prev_agreements: dict,
3      curr_agreements: dict,
4      round_num: int,
5      max_rounds: int = 3
6  ) → tuple[bool, str]:
7
8      # 规则 1: 达到最大轮次
9      if round_num ≥ max_rounds:
10         return True, "max_rounds"
11
12     # 规则 2: 所有分歧字段收敛
13     if all(v ≥ 0.67 for v in curr_agreements.values()):

```

```

14         return True, "convergence"
15
16     # 规则 3: Plateau 检测—本轮无提升
17     total_improvement = sum(
18         curr_agreements[f] - prev_agreements[f]
19         for f in curr_agreements
20     )
21     if total_improvement ≤ 0.0:
22         return True, "plateau"
23
24     return False, "continue"

```

- **convergence**: 正常收敛, Full Arbitrator 输出共识标签
- **max_rounds** / **plateau**: 辩论失败, 对应字段标记为 **uncertain**, 记录 disagreement 模式供分析 (L3f)

5.7 Full Arbitrator 与 uncertain 输出

当辩论终止后, Full Arbitrator 对每个字段:

- 若 $\text{agreement} \geq 0.67$: 输出多数标签, 记录置信度
- 若 $\text{agreement} < 0.67$ (辩论失败): **label=null, uncertain=true**

处理 **fix_type** 缺失问题 (见 1.2 节, 60.5% 有标签):

fix_type 在数据集中缺失率高达 39.5%, 这既是数据问题也是任务难度问题。Full Arbitrator 对 **fix_type** 使用更低的 **uncertain** 阈值 (0.5 而非 0.67), 并在输出中单独标记 **fix_type_inferred=true** 以示区分。

六、与现有代码的映射

6.1 pipeline.py 改动摘要

现有方法	新方法	变化说明
_run_filtering : 3 Filter Agent 多数投票	_run_stage2_verify_v2 : 四路并行 + 置信度门控	新增四类专项 Analyzer
_run_classification : Symptom→RootCause 串行	_run_stage3_classify_v2 : 三 Annotator 并行 + IRR	并行取代串行

现有方法	新方法	变化说明
单一 Critic（事后检查）	Validity Critic （ Stage 2 ） + SOP Validator （Stage 3）	职责拆分
单一 Arbitrator	Fast + Full 双轨 Arbitrator	无分歧时跳过辩论
无不确定性字段	<code>uncertain_fields</code> 、 <code>review_flag</code>	新增

6.2 新增文件

```
1  autoempirical_mas/
2  |— pipeline.py           # 新增 _run_stage2_verify_v2,
   |_run_stage3_classify_v2
3  |— prompts.py           # 新增 Stage 2/3 专项 prompt (SOP 格式)
4  |— agents.py            # 新增 IRRDetector, DebateOrchestrator, SOPValidator
5  |— schemas.py           # 新增 Stage2Result, Stage3Result, AnnotatorOutput
6  |_ sop/
7     |— symptom_sop.yaml
8     |— root_cause_sop.yaml
9     |— bug_type_sop.yaml
10    |_ taxonomy_aliases.json
```

七、消融实验设计

7.1 实验变体

变体	描述	验证目标
<code>single_agent</code>	单 LLM 一次性输出全部标签	基础 baseline
<code>mas_v1</code>	现 有 串 行 MAS (Symptom→RootCause→Critic→Arbitrator)	与新设计直接对比
<code>stage2_text_only</code>	Stage 2 仅用 Text Analyzer	验证四路证据必要性 (L2a)
<code>stage2_no_gate</code>	Stage 2 无置信度门控	验证门控对效率/质量影响
<code>stage3_no_irr</code>	Stage 3 三视角并行但无 IRR，直接多数投票	验证 IRR-Debate 必要性 (L3c)
<code>stage3_no_sop</code>	Stage 3 有 IRR 但无 SOP Validator	验证 SOP 约束必要性 (L3b)

变体	描述	验证目标
stage3_no_debate	Stage 3 有 IRR 检测但无辩论，直接 uncertain	验证辩论的贡献
full_mas_v2	完整新设计 Stage 2 + Stage 3	最终系统

7.2 评估指标

Stage 2:

指标	说明
Precision / Recall / F1 (Accepted)	核心分类质量
False Positive Rate	非 bug 被接受比例，对应 L1
Uncertain Calibration	uncertain 标记中人工确认难以判断的比例
API calls / record	计算成本

Stage 3:

指标	说明
Per-field F1 (symptom / root_cause / bug_type / fix_type)	各字段分类质量
Symptom-RootCause Consistency Rate	人工抽样 100 条，因果逻辑自治比例，对应 L3a
Taxonomy Violation Rate	SOP Validator 拒绝次数/总标签数，对应 L3b
IRR Trigger Rate	进入辩论的记录比例
Debate Termination Distribution	convergence / plateau / max_rounds 各占比
Uncertain Field Precision	uncertain 字段中人工标注确实困难的比例
Kappa (agent vs. gold)	参考 OLAF (arXiv:2512.15979) 的 calibration 要求

7.3 数据划分

类比人工实证研究的 IRR 流程:

子集	规模	用途
开发集 (50%)	~950 条	Prompt 调优、消融实验、超参数选择
讨论集 (25%)	~475 条	完整流水线运行，人工审查 uncertain 案例

子集	规模	用途
测试集（25%）	~475 条	最终评估，不参与任何调优

分层采样策略：

```
1 # 按 paper_id 和 text_availability 分层，确保各 paper 在三子集中均有代表
2 # text_availability 分三层：
3 #   "rich"      : body 长度 > 500 字符 (ase2021/ase2022/icse2022_perf/icse2023 类)
4 #   "short"     : body 长度 ≤ 500 字符 (fse2023/icse2022_wechat/icse2024/fse2021
5 #   "absent"    : 99 条无 body 记录
6 stratify_columns = ["paper_id", "text_availability"]
```

八、参考文献

引用	论文	使用点
[Dunivin 2026]	Self-reflection in Automated Qualitative Coding. arXiv:2601.09905	L1: 假阳性率 8–54%；置信度门控依据
[Laiq 2025]	Automatic Techniques for Issue Report Classification. arXiv:2505.01469	L3: 训练数据噪声；L2a: 忽略外部证据
[Andrade 2025]	Empirical Study on Bug Report Classification. arXiv:2503.00660	L2b: feature request 误判为 bug
[Gon 2026]	Automated Root-Cause Subclassification for Invalid Bug Reports. arXiv:2605.17561	L2c: invalid 子类 F1 低至 0.00–0.29
[OLAF 2026]	OLAF: Towards Robust LLM-Based Annotation. arXiv:2512.15979	L2: 非确定性；L2d: 不确定性量化；kappa 评估
[Baltes 2025]	Guidelines for Empirical Studies involving LLMs. arXiv:2508.15503	L2: 提示敏感性
[Martinez 2025]	LLMs in Thematic Analysis. arXiv:2510.18456	L6: 领域推理不足
[Chen 2025]	LLM Capability in Decomposing Bug Reports. arXiv:2504.20911	L6; SWE-MIMIC-Bench 证据提取 schema
[Díaz 2021]	Inter-rater Reliability in Grounded Theory Studies. arXiv:2107.11449	IRR 设计依据； $\kappa \geq 0.6$ 阈值
[Orchestration 2024]	AI Annotation Orchestration: LLM Verifiers. arXiv:2511.09785	orchestration 提升 kappa 58% ；IRR-debate 直接实验支持

引用	论文	使用点
[Tajik 2026]	Disagreement as Data: Reasoning Trace Analytics. arXiv:2601.12618	L3f: disagreement 作为有价值过程数据
[Borchers 2025]	Temperature and Persona Shape LLM Consensus. arXiv:2507.11198	L3e: 温度和 persona 对 consensus 的影响
[Flow-of-Action 2025]	Flow-of-Action: SOP Enhanced LLM MAS for RCA. arXiv:2502.08224	P4 SOP 约束; SOP 文档格式; RCA 准确率 64% vs 35.5%
[CollabEval 2026]	CollabEval: Enhancing LLM-as-a-Judge. arXiv:2603.00993	Debate Protocol; 3 轮 上 限 ; plateau detection
[Multi-role 2024]	Multi-role Consensus through LLMs for Vuln. Detection. arXiv:2403.14274	三视角并行框架; F1 +16.13%
[Threads 2026]	From Threads to Trajectories. arXiv:2604.25880	P1 证据驱动; Evidence Agent 设计
[MetaGPT 2024]	MetaGPT. arXiv:2308.00352	Publish-subscribe 消息池; SOP 驱动 MAS